

Dataset: [Employee productivity Dataset](#)

Q1. Handle Missing Values (Basic Cleaning)

Fill missing values using the following:

- Age → Median
- Salary → Mean
- Hours_Worked_Per_Week → Median
- Performance_Score → Mean

Display the dataset after handling missing values.

Ans:-

```
import pandas as pd

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# Fill missing values
df['Age'] = df['Age'].fillna(df['Age'].median())

df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(
    df['Hours_Worked_Per_Week'].median()
)

df['Performance_Score'] = df['Performance_Score'].fillna(
    df['Performance_Score'].mean()
)

# Display dataset
print(df)

# Verify missing values
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())
```

	Employee_ID	Name	Department	Age	Gender	Work_Mode	Join_Date	\
0	E001	Amit Sharma	IT	25.0	Male	Remote	15-01-2021	
1	E002	Riya Patel	HR	30.0	Female	Hybrid	15-02-2020	
2	E003	John Mathew	Finance	29.0	Male	Onsite	10-03-2022	
3	E004	Sneha Joshi	IT	28.0	Female	Remote	22-04-2021	
4	E005	Arjun Verma	Marketing	35.0	Male	Hybrid	18-05-2020	
..	...	...	...	...	...	...	...	
245	E006	Neha Singh	HR	27.0	Female	Remote	01-06-2023	
246	E007	Rahul Das	Finance	40.0	Male	Onsite	07-07-2019	
247	E008	Pooja Mehta	Marketing	29.0	Female	Hybrid	12-08-2022	
248	E009	Sameer Khan	IT	31.0	Male	Remote	30-09-2021	
249	E010	Kiran Nair	HR	26.0	Female	Onsite	05-10-2023	

	Hours_Worked_Per_Week	Projects_Completed	Performance_Score	\
0	40.0	5	4.5000	
1	38.0	4	4.0000	
2	45.0	6	4.3125	
3	41.0	5	4.2000	
4	50.0	7	4.8000	
..	...	...	...	
245	36.0	3	3.9000	
246	48.0	8	4.7000	
247	42.0	6	4.3000	
248	41.0	5	4.1000	
249	37.0	4	4.3125	

	Salary	Location
0	50000.000000	Pune
1	45000.000000	Mumbai
2	60000.000000	Bangalore
3	52000.000000	Delhi
4	50666.666667	Pune
..	...	...
245	42000.000000	Mumbai
246	65000.000000	Delhi
247	48000.000000	Bangalore
248	51000.000000	Pune
249	43000.000000	Mumbai

[250 rows x 12 columns]

Missing Values After Cleaning:

Employee_ID	0
Name	0
Department	0
Age	0
Gender	0
Work_Mode	0
Join_Date	0
Hours_Worked_Per_Week	0
Projects_Completed	0
Performance_Score	0
Salary	0
Location	0

dtype: int64

Q2. Label Encoding

Convert these columns into numeric values using Label Encoding:

- Gender
- Department

Show the updated columns.

Ans:-

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# (Optional) Handle missing values from Q1
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(
    df['Hours_Worked_Per_Week'].median()
)
df['Performance_Score'] = df['Performance_Score'].fillna(
    df['Performance_Score'].mean()
)

# Create LabelEncoder object
le = LabelEncoder()

# Apply Label Encoding
df['Gender'] = le.fit_transform(df['Gender'])
df['Department'] = le.fit_transform(df['Department'])

# Display updated columns
print(df[['Gender', 'Department']])

# Display encoding mapping
print("\nUnique Encoded Values:")
print("Gender:", df['Gender'].unique())
print("Department:", df['Department'].unique())
```

	Gender	Department
0	1	2
1	0	1
2	1	0
3	0	2
4	1	3
..	...	...
245	0	1
246	1	0
247	0	3
248	1	2
249	0	1

[250 rows x 2 columns]

Unique Encoded Values:

Gender: [1 0]

Department: [2 1 0 3]

Q3. One-Hot Encoding

Apply One-Hot Encoding on:

- Work_Mode
- Location

Display the dataset and check how many new columns are created.

Ans:-

```
import pandas as pd

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# Q1: Handle Missing Values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(
    df['Hours_Worked_Per_Week'].median()
)
df['Performance_Score'] = df['Performance_Score'].fillna(
    df['Performance_Score'].mean()
)

# Q3: One-Hot Encoding
df_encoded = pd.get_dummies(
    df,
    columns=['Work_Mode', 'Location'],
    dtype=int
)

# Display dataset
print(df_encoded)

# Count new columns created
original_columns = len(df.columns)
new_columns = len(df_encoded.columns)

print("\nOriginal Number of Columns :", original_columns)
print("New Number of Columns      :", new_columns)
print("New Columns Created        :", new_columns - original_columns + 2)
```

	Employee_ID	Name	Department	Age	Gender	Join_Date	\
0	E001	Amit Sharma	IT	25.0	Male	15-01-2021	
1	E002	Riya Patel	HR	30.0	Female	15-02-2020	
2	E003	John Mathew	Finance	29.0	Male	10-03-2022	
3	E004	Sneha Joshi	IT	28.0	Female	22-04-2021	
4	E005	Arjun Verma	Marketing	35.0	Male	18-05-2020	
...	...	...	...	...	...	...	
245	E006	Neha Singh	HR	27.0	Female	01-06-2023	
246	E007	Rahul Das	Finance	40.0	Male	07-07-2019	
247	E008	Pooja Mehta	Marketing	29.0	Female	12-08-2022	
248	E009	Sameer Khan	IT	31.0	Male	30-09-2021	
249	E010	Kiran Nair	HR	26.0	Female	05-10-2023	

	Hours_Worked_Per_Week	Projects_Completed	Performance_Score	\
0	40.0	5	4.5000	
1	38.0	4	4.0000	
2	45.0	6	4.3125	
3	41.0	5	4.2000	
4	50.0	7	4.8000	
...	...	...	...	
245	36.0	3	3.9000	
246	48.0	8	4.7000	
247	42.0	6	4.3000	
248	41.0	5	4.1000	
249	37.0	4	4.3125	

	Salary	Work_Mode_Hybrid	Work_Mode_Onsite	Work_Mode_Remote	\
0	50000.000000	0	0	1	
1	45000.000000	1	0	0	
2	60000.000000	0	1	0	
3	52000.000000	0	0	1	
4	50666.666667	1	0	0	
...	...	...	...	...	
245	42000.000000	0	0	1	
246	65000.000000	0	1	0	
247	48000.000000	1	0	0	
248	51000.000000	0	0	1	
249	43000.000000	0	1	0	

	Location_Bangalore	Location_Delhi	Location_Mumbai	Location_Pune
0	0	0	0	1
1	0	0	1	0
2	1	0	0	0
3	0	1	0	0
4	0	0	0	1
..	...	...	...	...
245	0	0	1	0
246	0	1	0	0
247	1	0	0	0
248	0	0	0	1
249	0	0	1	0

[250 rows x 17 columns]

Original Number of Columns : 12
 New Number of Columns : 17
 New Columns Created : 7

Check Number of New Columns Created

```
print("\nEncoded Columns:")
for col in df_encoded.columns:
    if "Work_Mode_" in col or "Location_" in col:
        print(col)
```

Encoded Columns:
 Work_Mode_Hybrid
 Work_Mode_Onsite
 Work_Mode_Remote
 Location_Bangalore
 Location_Delhi
 Location_Mumbai
 Location_Pune

Q4. Normalization (Min-Max Scaling)

Normalize the following columns:

- Salary
- Hours_Worked_Per_Week

Use MinMaxScaler and display results.

Ans:-

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# Q1: Handle Missing Values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(
    df['Hours_Worked_Per_Week'].median()
)
df['Performance_Score'] = df['Performance_Score'].fillna(
    df['Performance_Score'].mean()
)

# Min-Max Scaling
scaler = MinMaxScaler()

df[['Salary_Normalized', 'Hours_Worked_Normalized']] = scaler.fit_transform(
    df[['Salary', 'Hours_Worked_Per_Week']]
)

# Display results
print(df[['Salary',
            'Salary_Normalized',
            'Hours_Worked_Per_Week',
            'Hours_Worked_Normalized']])
```

	Salary	Salary_Normalized	Hours_Worked_Per_Week	\
0	50000.000000	0.347826	40.0	
1	45000.000000	0.130435	38.0	
2	60000.000000	0.782609	45.0	
3	52000.000000	0.434783	41.0	
4	50666.666667	0.376812	50.0	
..	...	...	...	
245	42000.000000	0.000000	36.0	
246	65000.000000	1.000000	48.0	
247	48000.000000	0.260870	42.0	
248	51000.000000	0.391304	41.0	
249	43000.000000	0.043478	37.0	

	Hours_Worked_Normalized
0	0.285714
1	0.142857
2	0.642857
3	0.357143
4	1.000000
..	...
245	0.000000
246	0.857143
247	0.428571
248	0.357143
249	0.071429

[250 rows x 4 columns]

Verification

```
print(df[['Salary_Normalized',  
         'Hours_Worked_Normalized']].min())  
  
print(df[['Salary_Normalized',  
         'Hours_Worked_Normalized']].max())
```

```
Salary_Normalized      0.0  
Hours_Worked_Normalized 0.0  
dtype: float64  
Salary_Normalized      1.0  
Hours_Worked_Normalized 1.0  
dtype: float64
```

Q5. Standardization (Scaling)

Apply StandardScaler on:

- Age
- Projects_Completed

Display the transformed values.

Ans:-

```
import pandas as pd  
from sklearn.preprocessing import StandardScaler  
  
# Load dataset  
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")  
  
# Q1: Handle Missing Values  
df['Age'] = df['Age'].fillna(df['Age'].median())  
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())  
df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(  
    df['Hours_Worked_Per_Week'].median()  
)  
df['Performance_Score'] = df['Performance_Score'].fillna(  
    df['Performance_Score'].mean()  
)  
  
# Apply StandardScaler  
scaler = StandardScaler()  
  
df[['Age_Standardized', 'Projects_Completed_Standardized']] = scaler.fit_transform(  
    df[['Age', 'Projects_Completed']])  
  
# Display transformed values  
print(df[['Age',  
         'Age_Standardized',  
         'Projects_Completed',  
         'Projects_Completed_Standardized']])
```

	Age	Age_Standardized	Projects_Completed	\
0	25.0	-1.172018		5
1	30.0	0.000000		4
2	29.0	-0.234404		6
3	28.0	-0.468807		5
4	35.0	1.172018		7
..	...	...		...
245	27.0	-0.703211		3
246	40.0	2.344036		8
247	29.0	-0.234404		6
248	31.0	0.234404		5
249	26.0	-0.937614		4

	Projects_Completed_Standardized
0	-0.211604
1	-0.916949
2	0.493742
3	-0.211604
4	1.199088
..	...
245	-1.622295
246	1.904433
247	0.493742
248	-0.211604
249	-0.916949

[250 rows x 4 columns]

Verification

```
print("Mean:")
print(df[['Age_Standardized',
          'Projects_Completed_Standardized']].mean())

print("\nStandard Deviation:")
print(df[['Age_Standardized',
          'Projects_Completed_Standardized']].std())
```

```
Mean:
Age_Standardized          1.598721e-17
Projects_Completed_Standardized  5.995204e-17
dtype: float64
```

```
Standard Deviation:
Age_Standardized          1.002006
Projects_Completed_Standardized  1.002006
dtype: float64
```

Q6. Compare Scaling Methods

Apply both:

- MinMaxScaler
- StandardScaler

On the Salary column.

Show both results side by side.

Ans:-

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# Handle missing values (Q1)
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

# Create scaler objects
minmax_scaler = MinMaxScaler()
standard_scaler = StandardScaler()

# Apply scaling on Salary
df['Salary_MinMax'] = minmax_scaler.fit_transform(df[['Salary']])

df['Salary_Standardized'] = standard_scaler.fit_transform(df[['Salary']])

# Display results side by side
print(df[['Salary', 'Salary_MinMax', 'Salary_Standardized']])
```

	Salary	Salary_MinMax	Salary_Standardized
0	50000.000000	0.347826	-0.097451
1	45000.000000	0.130435	-0.828333
2	60000.000000	0.782609	1.364312
3	52000.000000	0.434783	0.194902
4	50666.666667	0.376812	0.000000
..	...	...	...
245	42000.000000	0.000000	-1.266862
246	65000.000000	1.000000	2.095194
247	48000.000000	0.260870	-0.389804
248	51000.000000	0.391304	0.048725
249	43000.000000	0.043478	-1.120685

[250 rows x 3 columns]

Verification Min-Max Scaled Salary

```
print("Min-Max Scaling Range:")
print(df['Salary_MinMax'].min())
print(df['Salary_MinMax'].max())
```

Min-Max Scaling Range:

0.0

1.0

Q7. Build Preprocessing Pipeline

Create a pipeline that:

- Applies encoding to categorical columns
- Applies scaling to numerical columns

Use:

- ColumnTransformer
- Pipeline

Ans:-

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# Handle missing values (Q1)
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(
    df['Hours_Worked_Per_Week'].median()
)
df['Performance_Score'] = df['Performance_Score'].fillna(
    df['Performance_Score'].mean()
)

# Categorical and Numerical Columns
categorical_cols = [
    'Gender',
    'Department',
    'Work_Mode',
    'Location'
]

numerical_cols = [
    'Age',
    'Hours_Worked_Per_Week',
    'Projects_Completed',
    'Performance_Score',
    'Salary'
]

# Preprocessing using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'),
         categorical_cols),

        ('num', StandardScaler(),
         numerical_cols)
    ]
)

# Create Pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor)
])

# Apply Pipeline
processed_data = pipeline.fit_transform(df)

# Display shape of transformed data
print("Shape of Processed Data:")
print(processed_data.shape)

# Display transformed data
print("\nProcessed Data:")
print(processed_data)
```

Shape of Processed Data:
(250, 18)

Processed Data:

```
[[ 0.          1.          0.          ... -0.21160368  0.68522446
 -0.09745089]
 [ 1.          0.          0.          ... -0.9169493  -1.14204076
 -0.82833257]
 [ 0.          1.          1.          ...  0.49374193  0.
 1.36431247]
 ...
 [ 1.          0.          0.          ...  0.49374193 -0.04568163
 -0.38980356]
 [ 0.          1.          0.          ... -0.21160368 -0.77658772
 0.04872545]
 [ 1.          0.          0.          ... -0.9169493  0.
 -1.12068525]]
```

View Feature Names After Transformation

```
feature_names = pipeline.named_steps['preprocessor'].get_feature_names_out()

print("\nFeature Names:")
print(feature_names)
```

Feature Names:

```
['cat__Gender_Female' 'cat__Gender_Male' 'cat__Department_Finance'
 'cat__Department_HR' 'cat__Department_IT' 'cat__Department_Marketing'
 'cat__Work_Mode_Hybrid' 'cat__Work_Mode_Onsite' 'cat__Work_Mode_Remote'
 'cat__Location_Bangalore' 'cat__Location_Delhi' 'cat__Location_Mumbai'
 'cat__Location_Pune' 'num__Age' 'num__Hours_Worked_Per_Week'
 'num__Projects_Completed' 'num__Performance_Score' 'num__Salary']
```

Convert Output to DataFrame

```
processed_df = pd.DataFrame(
    processed_data,
    columns=feature_names
)

print(processed_df.head())
```

	cat__Gender_Female	cat__Gender_Male	cat__Department_Finance	\
0	0.0	1.0	0.0	
1	1.0	0.0	0.0	
2	0.0	1.0	1.0	
3	1.0	0.0	0.0	
4	0.0	1.0	0.0	

	cat__Department_HR	cat__Department_IT	cat__Department_Marketing	\
0	0.0	1.0	0.0	
1	1.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	1.0	0.0	
4	0.0	0.0	1.0	

	cat__Work_Mode_Hybrid	cat__Work_Mode_Onsite	cat__Work_Mode_Remote	\
0	0.0	0.0	1.0	
1	1.0	0.0	0.0	
2	0.0	1.0	0.0	
3	0.0	0.0	1.0	
4	1.0	0.0	0.0	

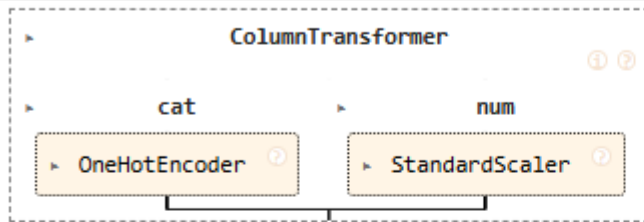
	cat__Location_Bangalore	cat__Location_Delhi	cat__Location_Mumbai	\
0	0.0	0.0	0.0	
1	0.0	0.0	1.0	
2	1.0	0.0	0.0	
3	0.0	1.0	0.0	
4	0.0	0.0	0.0	

	cat__Location_Pune	num__Age	num__Hours_Worked_Per_Week	\
0	1.0	-1.172018	-0.411220	
1	0.0	0.000000	-0.868132	
2	0.0	-0.234404	0.731059	
3	0.0	-0.468807	-0.182765	
4	1.0	1.172018	1.873338	

	num__Projects_Completed	num__Performance_Score	num__Salary
0	-0.211604	0.685224	-0.097451
1	-0.916949	-1.142041	-0.828333
2	0.493742	0.000000	1.364312
3	-0.211604	-0.411135	0.194902
4	1.199088	1.781584	0.000000

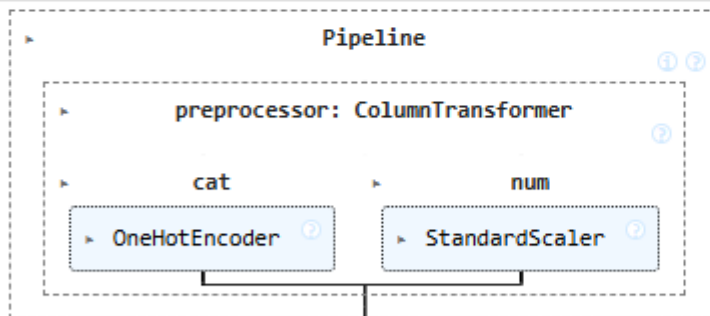
ColumnTransformer

```
ColumnTransformer([
    ('cat', OneHotEncoder(), categorical_cols),
    ('num', StandardScaler(), numerical_cols)
])
```



Pipeline

```
Pipeline([
    ('preprocessor', preprocessor)
])
```



Pipeline with Machine Learning Model

```
from sklearn.linear_model import LinearRegression

model_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])
```

Q8. Apply Pipeline

Apply the pipeline on the dataset.

Display:

- Transformed dataset
- Shape of final dataset

Ans:-

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Load dataset
df = pd.read_csv("F:\PW Assignment- 7 Python\Python Assignment- 6\employee_productivity_dataset.csv")

# Handle Missing Values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] = df['Hours_Worked_Per_Week'].fillna(
    df['Hours_Worked_Per_Week'].median()
)
df['Performance_Score'] = df['Performance_Score'].fillna(
    df['Performance_Score'].mean()
)

# Define columns
categorical_cols = [
    'Gender',
    'Department',
    'Work_Mode',
    'Location'
]

numerical_cols = [
    'Age',
    'Hours_Worked_Per_Week',
    'Projects_Completed',
    'Performance_Score',
    'Salary'
]

# Column Transformer
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'),
         categorical_cols),

        ('num', StandardScaler(),
         numerical_cols)
    ]
)
```

```

# Pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor)
])

# Apply Pipeline
transformed_data = pipeline.fit_transform(df)

# Get Feature Names
feature_names = pipeline.named_steps[
    'preprocessor'
].get_feature_names_out()

# Convert to DataFrame
transformed_df = pd.DataFrame(
    transformed_data.toarray()
    if hasattr(transformed_data, "toarray")
    else transformed_data,
    columns=feature_names
)

# Display transformed dataset
print("Transformed Dataset:")
print(transformed_df)

# Display shape
print("\nFinal Dataset Shape:")
print(transformed_df.shape)

```

Transformed Dataset:

	cat__Gender_Female	cat__Gender_Male	cat__Department_Finance	\
0	0.0	1.0	0.0	
1	1.0	0.0	0.0	
2	0.0	1.0	1.0	
3	1.0	0.0	0.0	
4	0.0	1.0	0.0	
..	...	...	...	
245	1.0	0.0	0.0	
246	0.0	1.0	1.0	
247	1.0	0.0	0.0	
248	0.0	1.0	0.0	
249	1.0	0.0	0.0	

	cat__Department_HR	cat__Department_IT	cat__Department_Marketing	\
0	0.0	1.0	0.0	
1	1.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	1.0	0.0	
4	0.0	0.0	1.0	
..	...	...	...	
245	1.0	0.0	0.0	
246	0.0	0.0	0.0	
247	0.0	0.0	1.0	
248	0.0	1.0	0.0	
249	1.0	0.0	0.0	

	cat__Work_Mode_Hybrid	cat__Work_Mode_Onsite	cat__Work_Mode_Remote	\
0	0.0	0.0	1.0	
1	1.0	0.0	0.0	
2	0.0	1.0	0.0	
3	0.0	0.0	1.0	
4	1.0	0.0	0.0	
..	...	...	...	
245	0.0	0.0	1.0	
246	0.0	1.0	0.0	
247	1.0	0.0	0.0	
248	0.0	0.0	1.0	
249	0.0	1.0	0.0	

	cat__Location_Bangalore	cat__Location_Delhi	cat__Location_Mumbai	\
0	0.0	0.0	0.0	
1	0.0	0.0	1.0	
2	1.0	0.0	0.0	
3	0.0	1.0	0.0	
4	0.0	0.0	0.0	
..	...	...	...	
245	0.0	0.0	1.0	
246	0.0	1.0	0.0	
247	1.0	0.0	0.0	
248	0.0	0.0	0.0	
249	0.0	0.0	1.0	

	cat__Location_Pune	num__Age	num__Hours_Worked_Per_Week	\
0	1.0	-1.172018	-0.411220	
1	0.0	0.000000	-0.868132	
2	0.0	-0.234404	0.731059	
3	0.0	-0.468807	-0.182765	
4	1.0	1.172018	1.873338	
..	...	...	...	
245	0.0	-0.703211	-1.325044	
246	0.0	2.344036	1.416426	
247	0.0	-0.234404	0.045691	
248	1.0	0.234404	-0.182765	
249	0.0	-0.937614	-1.096588	

	num__Projects_Completed	num__Performance_Score	num__Salary
0	-0.211604	0.685224	-0.097451
1	-0.916949	-1.142041	-0.828333
2	0.493742	0.000000	1.364312
3	-0.211604	-0.411135	0.194902
4	1.199088	1.781584	0.000000
..	...	...	...
245	-1.622295	-1.507494	-1.266862
246	1.904433	1.416131	2.095194
247	0.493742	-0.045682	-0.389804
248	-0.211604	-0.776588	0.048725
249	-0.916949	0.000000	-1.120685

[250 rows x 18 columns]

Final Dataset Shape:
(250, 18)

Q9. Conceptual Question

Why is scaling important in Python?

Ans:-

Scaling is the process of transforming numerical features so that they are on a similar scale. It is important in Python, especially in **Machine Learning**, because different features may have very different ranges of values.

For example:

Age = 25

Salary = 50000

Here, Salary has a much larger value than Age. Without scaling, machine learning algorithms may give more importance to Salary and less importance to Age.

Importance of Scaling

1. Improves Model Performance

- Many algorithms such as KNN, SVM, Logistic Regression, and Neural Networks perform better when features are scaled.

2. Faster Convergence

- Scaling helps optimization algorithms like Gradient Descent reach the optimal solution faster.

3. Prevents Feature Dominance

- Large-valued features do not overpower smaller-valued features.

4. Improves Distance Calculations

- Algorithms that use distance measures (KNN, K-Means) require scaling for accurate results.

5. Ensures Fair Comparison

- All features contribute equally to the model.

Common Scaling Techniques

1. Min-Max Scaling

Transforms values to a range between 0 and 1.

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

2. Standardization (Z-Score Scaling)

Transforms data to have:

- Mean = 0
- Standard Deviation = 1

$$Z = \frac{X - \mu}{\sigma}$$

Example

Before Scaling:

Age	Salary
25	50000
30	60000

After Scaling:

Age	Salary
-0.85	-0.72
0.85	0.72

Q10. Conceptual Question

Why do we convert categorical data into numerical form?

Ans:-

Machine Learning algorithms work with mathematical calculations and cannot directly understand text or categorical values such as:

Gender = Male, Female

Department = IT, HR, Finance

Work_Mode = Remote, Office, Hybrid

Therefore, categorical data must be converted into numerical form before training a machine learning model.

Reasons for Converting Categorical Data

1. Machine Learning Algorithms Require Numbers

Most algorithms perform operations such as:

- Addition
- Subtraction
- Distance calculation
- Matrix multiplication

These operations can only be performed on numerical data.

Example:

<u>Male,</u>	<u>Female</u>
1	0

2. Improves Model Training

Numerical values allow algorithms to process data efficiently and learn patterns from the dataset.

Example:

Department

IT
HR
Finance

After encoding:

IT → 2
HR → 1
Finance → 0

3. Enables Mathematical Calculations

Algorithms like:

- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Logistic Regression
- Neural Networks

require numerical inputs for calculations.

4. Makes Data Suitable for Feature Engineering

Encoded categorical data can be combined with other numerical features and used in predictive models.

Common Encoding Techniques

1. Label Encoding

Assigns a unique number to each category.

Example:

Department	Encoded Value
------------	---------------

Finance	0
---------	---

HR	1
----	---

IT	2
----	---

Marketing	3
-----------	---

2. One-Hot Encoding

Creates separate binary columns for each category.

Example:

Work_Mode	Hybrid	Office	Remote
-----------	--------	--------	--------

Hybrid	1	0	0
--------	---	---	---

Office	0	1	0
--------	---	---	---

Remote	0	0	1
--------	---	---	---

Example

Before Encoding:

Gender Department

Male	IT
Female	HR

After Encoding:

Gender Department

1	2
0	1